SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

Balancia introduces an AI-powered management platform designed especially for project managers and small to medium-sized business owners.

The main problem is that managers spend too much time micro-managing employees and balancing different tasks, which reduces productivity. Hence, this system is designed to automate task planning and monitoring using a Generative Language Model (GLM).

1.1. Purpose:

The system analysis document highlights the system design, technical scope, and design-related decision behind Balancia.

The key elements of system that has been covered are as followed:

1. ***Architecture**:*
   Balancia uses a modern, serverless web architecture. The front end and API connections were built with Next.js 15, and persistent data and authentication is handled via Supabase. The AI reasoning (Z.ai's GLM) operates independently accessed via API.
2. ***Data Flows:***
   This documentation shows show unstructured input(The Admin's vague project goals) flows through the AI layer, how it is parsed, and queried using the data collected from the employees and routed and reflected as tasks on the employee side.
3. ***Model Process:***
   The model shows the end-to-end workflow of Balancia's core loop. How it breaks down complex objectives, matching it to the correct employee based on suitability. How it tracks work execution and how any occurring Friction is presented and resolved.
4. ***Role of Reference:***
   This document serves as the technical blueprint for developers, testers, which helps them understand the intended system interactions, database schemas and what constraints the solution has, and mechanisms surrounding how the LLM is integrated into the core product.

1.2. Background:

In fast-paced environments such as SME development teams and intellectual service based businesses such as marketing agencies without a large enough headcount that it justifies hiring someone to coordinate all these operations and that aren't small enough that it makes delegation easy and simple suffer from friction between executive intentions and technical execution/ instructions. Managers and executives can establish higher level, abstract objectives but breaking them down into what actionable tasks it requires to be completed, monitoring team capacity and real-time updates about what each member is working on and assigning work based on who is available and what would be the most efficient use of time is extremely tedious

© UMHackathon 2026

and manual, requiring a headcount just to help all the parts of the business operate towards a shared goal and often require many stand up meetings, paperwork, updates which take away precious time from the member's ability to actually produce value for the organization. This traditional approach as we can see results in misallocated resources in way of time, hard and manual to gather data on productivity and burnout from just trying to coordinate and work as a unit.

1. Previous Version:

- *In recent years, workplaces have relied on an ecosystem of fragmented products, like Jira, Trello combined with other communication channels of choice like Microsoft Team. Making a vague goal into actionable tickets relies on the manager's intuition and foresight. Resolving friction requires the kind of communication like meetings that halt the team's productivity.*

2. Changes in Major Architectural Components

- *Balancia shifts the architecture by integrating AI into this CRUD ticketing environment. We implement an AI reasoning layer to capture unstructured inputs and transform it into relational data.*

3. New capabilities that are only introduced in this

- *Automated Goal Decomposition: The AI automatically turn vague prompts into structured and actionable sub tasks.*
- *Skill Routing: Balenciaga will cross-reference the task against data of what the workforce is up to and that task assignments are well thought out concerning worker technical proficiency.*

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Business Owner /Manager	• *Inputs vague business goals (e.g. "launch website by next month")* • *Uploads employee resumes, data, and company documents* • *Monitors overall performance through dashboard*	• *Dashboard with clear overview of progress (goal tracking, monthly reports)* • *Ability to input vague goals for the system to break into tasks automatically* • *Gantt chart to visualize timeline* • *Recommendations for hiring or outsourcing* • *Integration with tools like Google Drive / Microsoft Teams*

© UMHackathon 2026

		• *Insights on productivity and business performance*
Employees	• *Receive tasks assigned by system* • *Update task progress and communicate with AI* • *Complete tasks within deadline*	• *Clear task breakdown and fair task allocation* • *Notifications and nudges to improve productivity* • *No overworking (balanced workload system)* • *Track time spent per task* • *Ability to answer system questions (skills, MBTI, interests)* • *Opportunities for training and incentives to improve performance*
AI System (GLM Engine)	• *Analyze employee data (resume, MBTI, skills)* • *Break vague goals into micro tasks* • *Allocate tasks to the most suitable employees* • *Track work rate and productivity*	• *Accurate data input (resumes, performance data)* • *Ability to evaluate efficiency and task suitability* • *Detect skill gaps to suggest hiring or outsourcing* • *Continuously optimize task allocation*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web Application (Admin & Employee interfaces)*
Architecture	*Serverless Architecture with AI microservice*

Balancia is mainly structured as a client-server web application to serve two distinct roles. For Admins, we have features for orchestration and analytics. For employees, we have task tracking, identity management and ticketing systems to resolve any blockers from goal accomplishment.

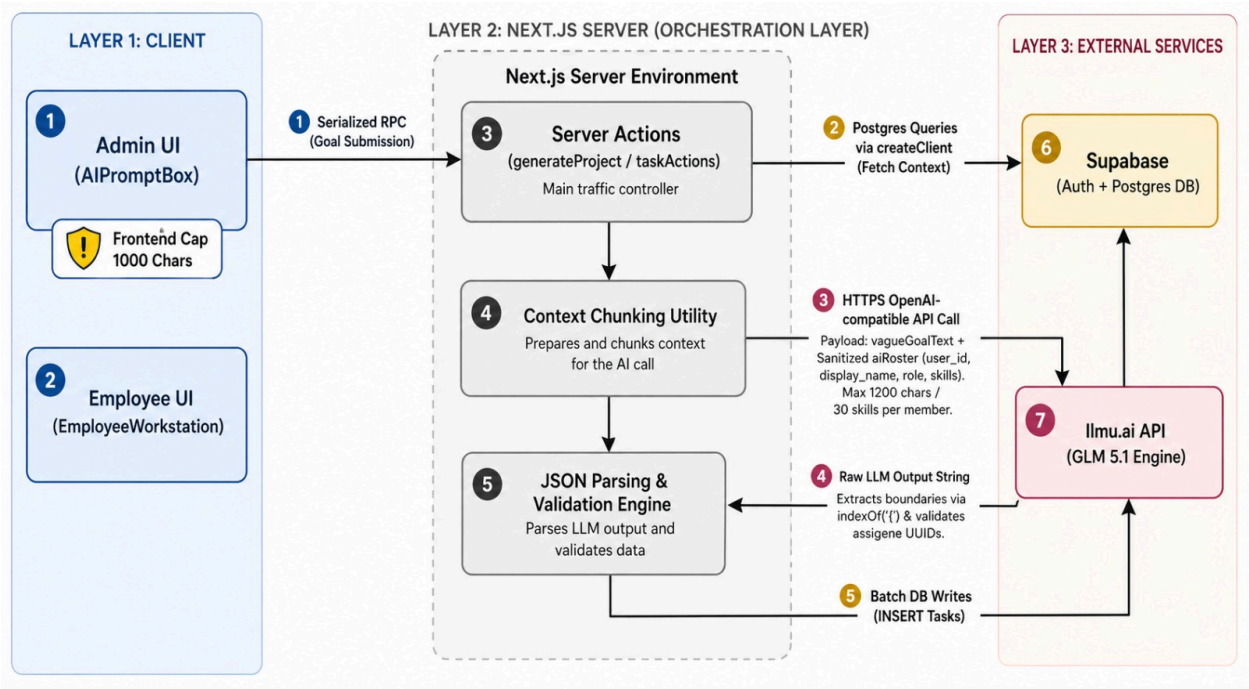
The interfaces communicate via Next.js Server Actions and API routes. Persistent data storage, and authentication are handled by a PostgreSQL database hosted on Supabase. To enable the core loop of creating and assigning tasks without manual input, Balancia lets all this complex reasoning to z.ai's GLM. All the backend components interact with this reasoning engine via asynchronous API calls.

2.1.2 LLM as Service Layer

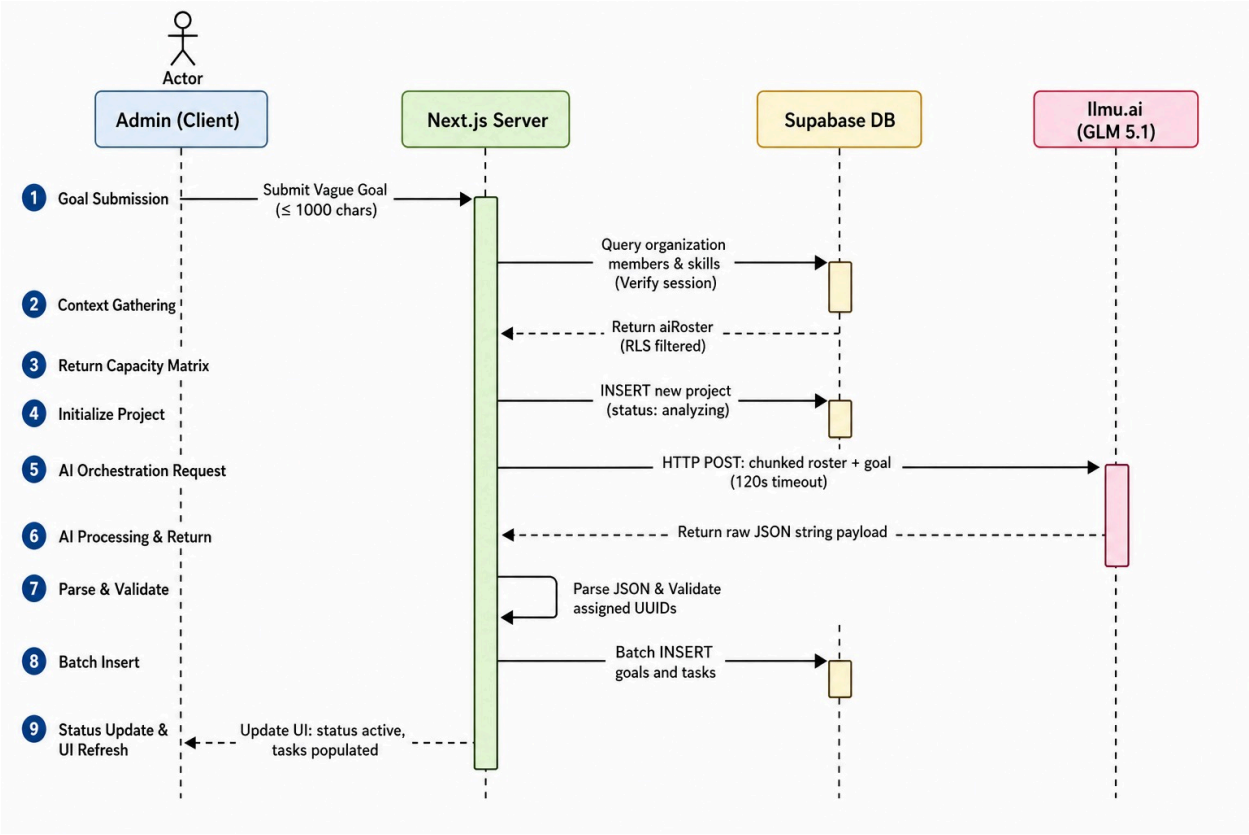
To enable the core loop of creating and assigning tasks without manual input, Balancia lets all this complex reasoning to z.ai's GLM. All the backend components interact with this reasoning engine via asynchronous API calls. The following Dependency diagram includes:

© UMHackathon 2026

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2. Technological Stack

2.2.1. Frontend

Next.js 15 (App router) was combined with Tailwind CSS and premade components from the Shadcn UI library. It gives a server-side rendered web interface, with ease to improve upon ui, handling our data representation heavy dashboards.

2.2.2. Backend

Next.js Server Actions and regular API routes that are tightly coupled with Supabase to handle the core application logic, and allows interaction with the ilmu-glm-5.1 model.

2.2.3. Database

PostgreSQL (hosted via supabase) to store relational data regarding organizations, members, tasks which all interrelate with each other. Supabase also allows for authentication of users and oversight on what data is each user allowed to access.

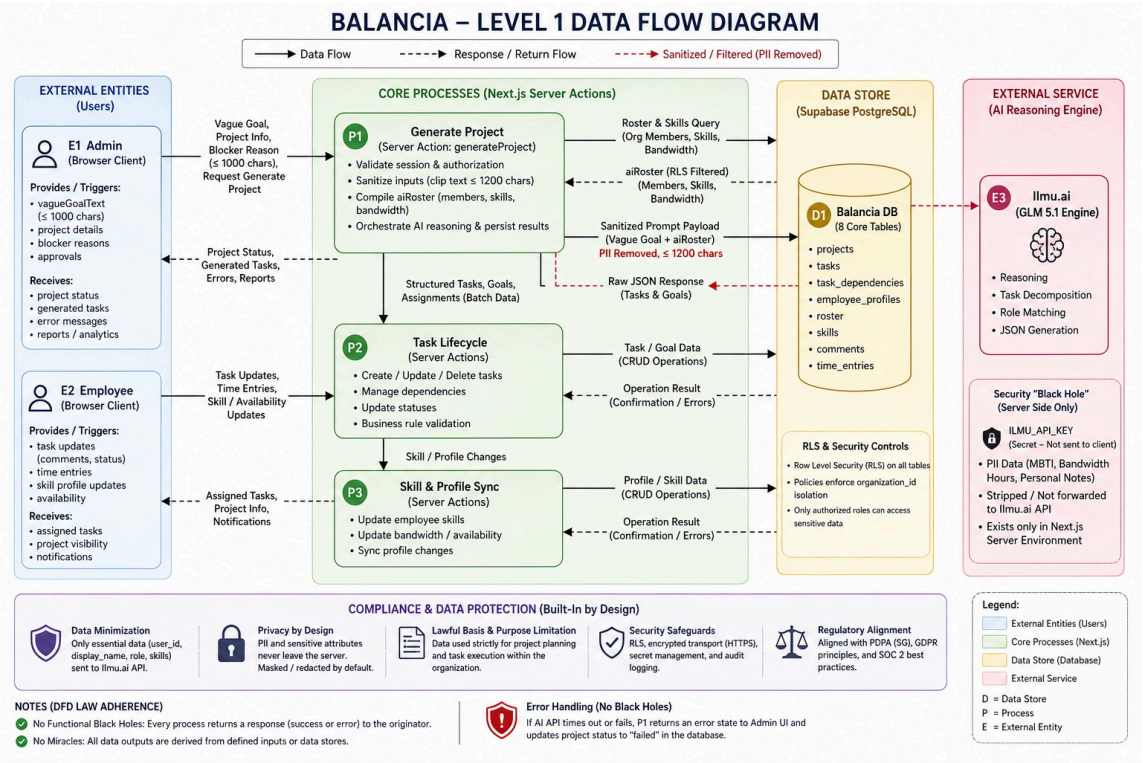
2.2.4. Cloud/Deployment

Vercel for seamless integration with Next.js frontend and quick redeployments.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored.

2.3.1.1. Data Flow Diagram (DFD)

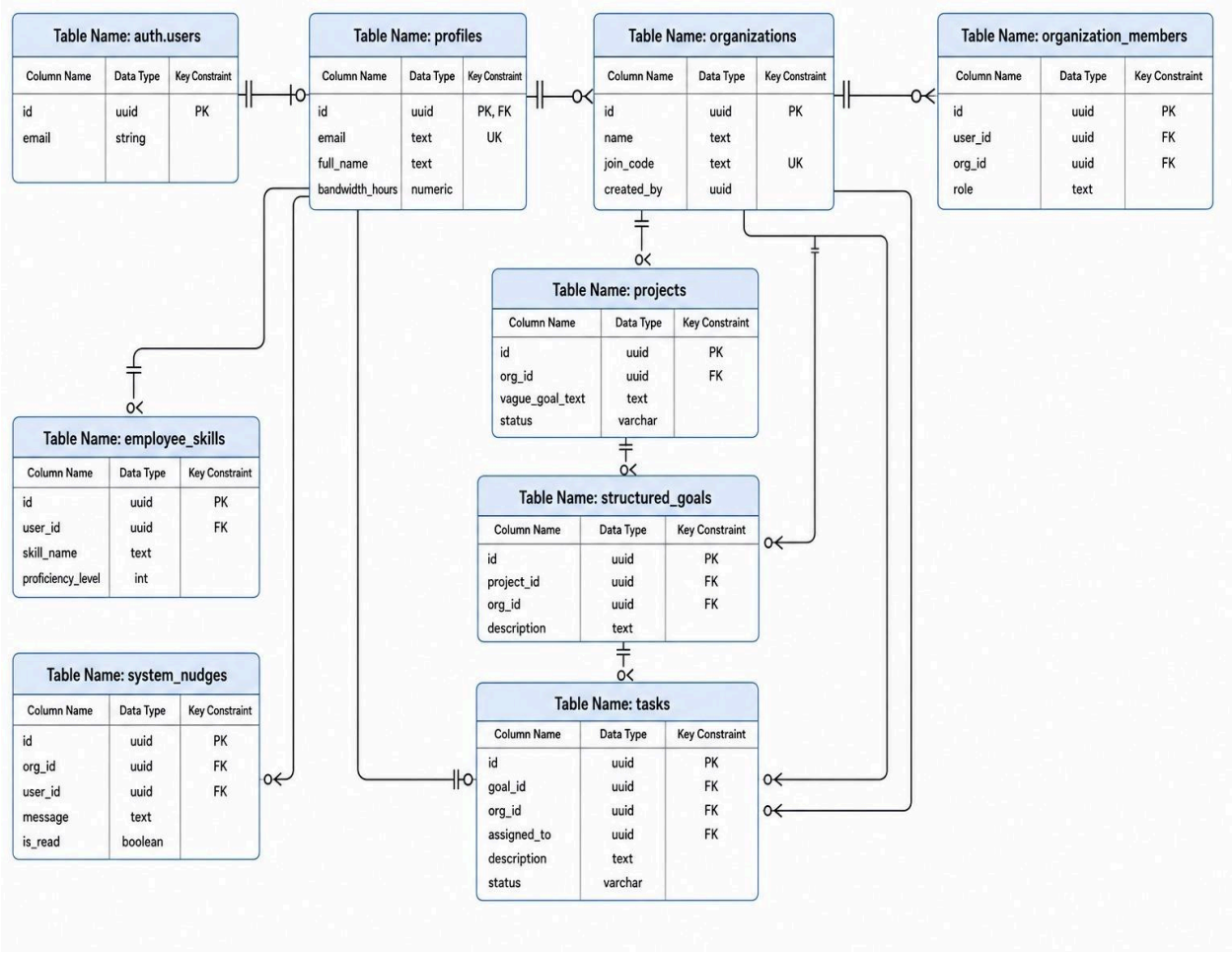


© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. Normalized Database Schema

a. Show the logical data model (ERD) with Database designed to 3NF form to ensure data integrity and relationships. [e.g. order items is a junction table that is resolving the M: N relationship between orders and menu items.



© UMHackathon 2026

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

3.1.1. *These are the features required as a MVP to be demo-ed.*

#	Features	Description
1	Goal Input & Task Breakdown	Users input vague goals (e.g. "build a landing page"), and the system uses AI to break them into structured micro tasks.
2	Smart Task Allocation	The system assigns tasks to employees based on skills, experience, and availability using resume and data analysis.
3	Productivity Tracking Dashboard	Dashboard shows task progress, work rate, time spent per task, and overall goal completion status.
4	SOS Friction Trigger	Employees can flag tasks if they are facing a problem which alerts admin.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	The system must support multiple organisations and high-volume task updates without crashing and delaying.	Using Next.js Server Actions and createServerClient prevents prolonged connection hangs.
Reliability	The system maintains data integrity so that there are no unauthorized data changes.	Double-Lock Security enforces ownership via application layer and the database layer via Row Level Security(RLS).
Maintainability	The system should be flexible so that rapid framework update, additional features and independent maintenance are allowed.	Database Access is centered in a single Supabase and server actions are separated into different files(taskActions.ts, auditActions.ts)

© UMHackathon 2026

Token Latency	*During LLM load, the system provides immediate AI responses (task allocation) by preventing "hanging" responses.*	*Implements a 120-second AbortController timeout and renders animated placeholders during AI calls.*
Cost Efficiency	*Maintain predictable AI token spend for each goal and protect against over usage of tokens by putting oversized user's inputs.*	*The system limits 1000 characters frontend on the vague goals(user's input) and utilizes clipForPrompt to truncate database text fields into small packs.*

3.3. Future Enhancements

a. **Tool Integration:** Balancia could integrate the system by connecting with the external tools to centralize the workflow. By integrating that, Balancia could automatically sync the files across multiple platforms such as Google Drive and Microsoft Teams. For example, the task that distributes can be linked to the document while the notifications and updates can be sent through the Microsoft Teams. This will further improve the efficiency and saving time as Balancia would be the control center of another tool.

b. **Predictive AI:** By implementing this feature, Balancia can help the teams to detect whether a task is taking too long to complete it. Thus, it could be helped to identify the potential delays occurring in the project. Hence, it could suggest reallocation of the tasks once again before these issues happen.

c. **Smart Hiring:** Balancia could help to detect the team capacity is insufficient by monitoring the workload and the capacity of a team. Based on the information given, Balancia will suggest whether to hire or outsource specific roles. Hence, it helps the organisations to scale efficiently without overloading employees.

d. **Personalize Engine:** Balancia's system will help to adapt to each employee as it will learn from the past performance to adjust the task difficulty and type whether these distributed tasks are matched to the employees' strength and skills.

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

Balancia will deploy a gradual rollout strategy for AI-related features such as task breakdown and allocation. New updates to the GLM logic will first be released to a small percentage of users (e.g. 5–10%) to validate task accuracy, latency (5–12 seconds), and system stability.

Once validated, the rollout will expand to all users. A/B testing will then be conducted to compare different AI decision strategies, such as task allocation logic or employee nudging methods. This is to evaluate which approach improves productivity and task completion rates better.

4.1.2. Emergency Rollback (ER) & Golden Release:

Since Balancia relies on AI-generated outputs, incorrect or low-quality task generation may occur. If critical issues are detected (e.g. unusable task plans or system failure), an Emergency Rollback will revert the system to a stable Golden Version.

Deployment is managed through Vercel to undergo quick redeployment to previous builds that are stable. This ensures minimal disruption to users and maintains system reliability during failures.

4.1.3. Priority Matrix:

4.1.3.1. P1 (Critical):

Trigger: System crash, corrupted data, or unusable AI-generated roadmap

Action: Immediate rollback to the Golden Version and disable AI generation

4.1.3.2. P2 (High):

Trigger: AI produces inaccurate or generic ("cookie-cutter") task plans

Action: Apply prompt refinement, restrict generation, and require stricter manual approval

4.1.3.3. P3 (Medium):

Trigger: AI response latency exceeds expected range (e.g.,>12 seconds)

Action: Optimize prompt size (compression/filtering) and queue requests

4.1.3.4. P4 (Low):

Trigger: UI overflow (too many tasks, layout issues)

Action: Apply UI fixes (Smart Canvas adjustments) in the next release

© UMHackathon 2026

4.2. Monitoring:

Balancia monitors system health continuously across its decoupled infrastructure, covering frontend performance, backend services, AI behavior, and productivity metrics.

Frontend & Edge Performance:
Vercel Analytics tracks Core Web Vitals, server-side rendering latency, and Server Action success rates to ensure smooth user experience.

Database & Security:
Supabase dashboard monitors query execution time, storage usage, and Row Level Security (RLS) violations to maintain performance and data isolation.

AI System Health:
Next.js server logs capture HTTP response status from the GLM API. Timeout events (e.g. exceeding 120,000ms) and failed responses are flagged for monitoring.

Productivity & System Metrics:
The system also tracks:

- *Task completion rate and delays*
- *Time spent per task (to detect overload or inefficiency)*
- *AI output quality (task breakdown and allocation accuracy)*
- *API latency and system uptime*

Trigger Mechanism:
If abnormal patterns are detected, such as high error rates, latency spikes, or a sudden drop in task completion, automated alerts are triggered. Logs from Next.js and Supabase are analyzed, and admins are notified via the dashboard for immediate action.

4.3. Assumptions:

Key Operational Conditions:
a. ***Data Accuracy:*** *Admins and employees ensure that organisational data (skills, roles, and bandwidth) in Supabase is accurate and updated regularly.*
b. ***Active Participation:*** *Employees consistently update their task progress and completion status to allow the AI to calculate real-time productivity.*
c. ***Goal Clarity:*** *Users provide natural language inputs with enough context for the GLM to perform effective intent parsing and task decomposition.*
Environmental Conditions:
d. ***Service Stability:*** *External cloud providers (**Z Ai GLM API, Supabase, and Vercel**) maintain their standard uptime and availability as per their Service Level Agreements (SLAs).*
e. ***Connectivity:*** *Users maintain a stable internet connection to ensure the Next.js client can send requests and receive **Optimistic UI** updates without latency issues.*

© UMHackathon 2026

4.4. External Dependencies:

Tools	*Purpose*	*Risks*
Z Ai GLM API	*It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output*	*High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]*
Supabase	*It is the primary relational data storage that handles authentication sessions via RLS.*	*High- Database can have downtime and it can stop the smoothness of the system. So, manage cloud infrastructure with automated daily backups and stateless REST clients and prevent connection pooling limits.*
Vercel Edge	*It keeps the website live, runs background code when the user clicks the button and automatically updates when code changes.*	*Medium: Latency may rise due to regional network disruptions. Serverless functions that scale automatically and globally distributed CDN for faster content delivery.*

© UMHackathon 2026

5. Project Management & Team Contributions

5.1. Project Timeline:

- ***Day 1 (19th April) – Idea Brainstorming:*** *The team discussed and explored different project ideas to identify a relevant problem solution.*
- ***Day 2 (20th April) – Planning & Task Distribution:*** *The idea was finalized through a Google Meet discussion. Roles and tasks were assigned to each team member to ensure efficient workflow.*
- ***Day 3 (21th April) – Data & Initial Development:*** *The ERD, sample data, and expected outputs were prepared. At the same time, basic interface development and documentation (PRD & SAD) were initiated.*
- ***Day 4 (22th April) – UI/UX Design:*** *The UI/UX design was completed to provide a clear structure for system interaction and guide frontend development. Draft UI/UX of the design are created to*
- ***Day 5–6 (23th – 24th April) – AI Integration & Testing:*** *AI (GLM) was integrated into the system. Testing and debugging were conducted to ensure correct task generation and system functionality.*
- ***Day 7-8 (25th -26th April) – Finalization:*** *Core system features were completed. The team continued refining and completing documentation. The demo video was prepared, and all documentation was finalized for submission.*

5.2. Team Members Role:

- ***Aung Myin Tun Soe – Backend Lead***

- *Architected/Configured the system and tech stack*
- *Performed Quality Assurance Testing and handled CI/ CD and deployment*
- *Worked with the GLM/z.ai's API*
- *Writing and providing data for System Analysis and Testing Analysis Documentation*

- ***La Pyae Aung – UX Designer***

- *Designs user experience and user flow.*
- *Writing Documentations on System Analysis, Risk Assessment & Mitigation Strategy and QA Testing .*
- *Creates a Use Case diagram.*
- *Helps on data architecture.*

- ***Wong Xin Yu – UI Designer***

- *Designs user interface layout, visuals, overall look and mascot of the system*
- *Showcase the UI/UX part, problems that solved and future features*
- *Writing documentation on future enhancements and recommendations*

- ***Tan Kher Er – Documentation & Backend Support Lead***

- *Prepares and organizes project documents and reports.*
- *Implemented UI/UX design into a fully functional website interface.*
- *Presented a live demonstration of the project*
- *Performed system testing and debugging to ensure the project runs smoothly*

- ***Thiri Moe Theint – Data & ERD***

- *Creates fake data and data architecture*
- *Writing Documentations on System Analysis, User Stories and Cases and QA Testing*
- *Performed Data Simulation and QA testing*
- *Presented Core functions MVP and other functions*

5.3. Recommendations:

- **Scalability:** The system can implement a caching mechanism to store frequently accessed data like the dashboard summaries and task lists. This will help to reduce the database load and improve the response time as well. Not only that, but also deploying the system on the cloud platforms to allow the system to handle increasing numbers of users efficiently and smoothly.
- **Performance:** For performance improvements, implemented the real-time updates by using the technologies to instantly reflect the task changes without requiring page refresh.
- **Reliability:** The system should include automated data backups and recovery mechanisms to prevent the data loss. In addition, proper error handling errors should be implemented as well to ensure the systems can be detected and resolved quickly while failures
- **Security:** The system should encrypt the sensitive data in transit and at rest to ensure data protection is safe while using the system.

© UMHackathon 2026